

1. Общие сведения об асинхронном программировании

1.1. Определение и базовые концепции

Асинхронное программирование — это парадигма, при которой выполнение длительных операций не блокирует основной поток выполнения программы. Вместо ожидания завершения операции, программа продолжает работу, а результат обрабатывается позднее через специальные механизмы (колбэки, промисы, `async/await`).

1.2. Синхронное vs Асинхронное выполнение

Синхронный подход (блокирующий):

javascript

```
const fs = require('fs');

// Каждая операция блокирует выполнение следующей
console.log('Начинаем чтение файла...');
const data1 = fs.readFileSync('file1.txt', 'utf8'); // Блокировка!
console.log('Файл 1 прочитан:', data1);

const data2 = fs.readFileSync('file2.txt', 'utf8'); // Блокировка!
console.log('Файл 2 прочитан:', data2);
```

```
console.log('Все файлы прочитаны');
```

Асинхронный подход (неблокирующий):

javascript

```
console.log('Начинаем чтение файла...');

fs.readFile('file1.txt', 'utf8', (err, data1) => {
  console.log('Файл 1 прочитан:', data1);
});
```

```
fs.readFile('file2.txt', 'utf8', (err, data2) => {
  console.log('Файл 2 прочитан:', data2);
});

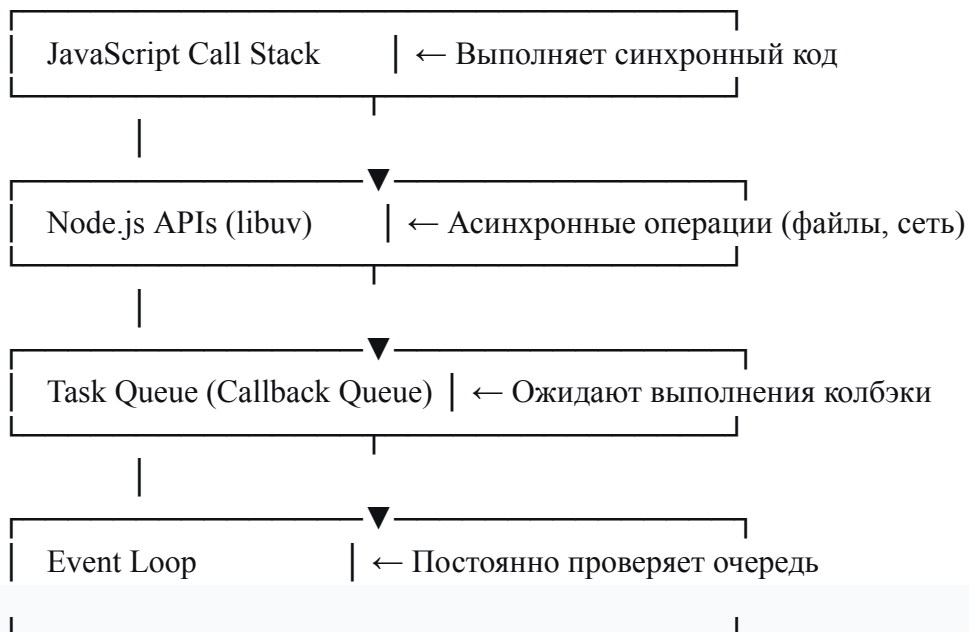
console.log('Чтение файлов запущено, продолжаем выполнение...');

// Вывод:
// Начинаем чтение файла...
// Чтение файлов запущено, продолжаем выполнение...
// Файл 1 прочитан: ... (когда загрузится)
// Файл 2 прочитан: ... (когда загрузится)
```

1.3. Как асинхронность работает в Node.js

Node.js использует однопоточный цикл событий (Event Loop), который позволяет обрабатывать множество асинхронных операций без создания новых потоков:

text



2. Преимущества и недостатки асинхронного программирования

2.1. Преимущества

Преимущество	Описание	Пример
Высокая производительность	Неблокирующий I/O позволяет обрабатывать тысячи запросов в одном потоке	Веб-сервер, обслуживающий 10,000+ одновременных подключений
Эффективное использование ресурсов	Нет необходимости создавать поток на каждый запрос	Экономия памяти и CPU
Отзывчивость UI	Интерфейс не "зависает" во время выполнения операций	Веб-приложения с плавной анимацией
Масштабируемость	Легко масштабировать горизонтально	Microservices архитектура
Гибкость	Возможность выполнять несколько операций параллельно	Promise.all, параллельные запросы к БД

Пример преимущества:

```
javascript
```

```
// Обработка 10,000 запросов без блокировки
```

```
const http = require('http');
```

```
const server = http.createServer((req, res) => {
  // Асинхронное чтение БД не блокирует другие запросы
  database.query('SELECT * FROM users', (err, users) => {
    res.writeHead(200);
    res.end(JSON.stringify(users));
  });
});
```

```
server.listen(3000); // Может обрабатывать тысячи запросов одновременно!
```

2.2. Недостатки

Недостаток	Описание	Пути решения
Сложность кода	Асинхронный код сложнее читать и поддерживать	async/await, хорошая архитектура
Callback Hell	Глубокая вложенность колбэков	Промисы, async/await
Сложная обработка ошибок	Ошибки могут "теряться" в асинхронных цепочках	try/catch, .catch(), глобальные обработчики
Race Conditions	Сложно управлять порядком выполнения	Promise.all, синхронизация
Отладка	Труднее отлаживать из-за асинхронного потока	async stack traces, debugger
Потенциальные утечки памяти	Забывтые обработчики событий	Правильное управление подписками

Пример недостатка (Callback Hell):

```
javascript
```

```
// Сложно читать и поддерживать
getUser(1, (err, user) => {
  if (err) throw err;
  getProfile(user.id, (err, profile) => {
    if (err) throw err;
```

```
getSettings(user.id, (err, settings) => {  
  if (err) throw err;  
  getNotifications(user.id, (err, notifications) => {  
    if (err) throw err;  
    // Четыре уровня вложенности!  
  });  
});  
});  
});  
});
```

3. Колбэки (Callbacks) в Node.js

3.1. Что такое колбэк?

Колбэк (callback) — это функция, которая передаётся как аргумент в другую функцию и вызывается после завершения асинхронной операции.

3.2. Паттерн error-first (ошибка-первая)

В Node.js принят стандарт error-first callbacks:

- Первый аргумент колбэка — ошибка (null, если ошибки нет)
- Последующие аргументы — результат операции

```
javascript
```

```
// Стандартный паттерн в Node.js  
fs.readFile('file.txt', 'utf8', (err, data) => {  
  if (err) {  
    // Обработка ошибки  
    console.error('Ошибка чтения:', err.message);  
    return;  
  }  
  // Успешное выполнение  
  console.log('Данные:', data);  
});
```

3.3. Примеры использования колбэков

Пример 1: Чтение файла

javascript

```
const fs = require('fs');

fs.readFile('config.json', 'utf8', (err, data) => {
  if (err) {
    console.error('Не удалось прочитать конфиг:', err.message);
    return;
  }

  try {
    const config = JSON.parse(data);
    console.log('Конфигурация загружена:', config);
    startApp(config);
  } catch (parseErr) {
    console.error('Ошибка парсинга JSON:', parseErr.message);
  }
});
```

Пример 2: HTTP-запрос

javascript

```
const https = require('https');

function fetchUserData(userId, callback) {
  const url = `https://api.example.com/users/${userId}`;

  https.get(url, (response) => {
    let data = '';

    response.on('data', (chunk) => {
      data += chunk;
    });

    response.on('end', () => {
      try {
        const user = JSON.parse(data);
        callback(null, user);
      } catch (err) {
        callback(err);
      }
    });
  });
}
```

```

    });
  }).on('error', (err) => {
    callback(err);
  });
}

// Использование
fetchUserData(123, (err, user) => {
  if (err) {
    console.error('Ошибка:', err.message);
    return;
  }
  console.log('Пользователь:', user);
});

```

3.4. Проблема "Callback Hell" (Ад колбэков)

javascript

```

// Пример "пирамиды гибели"
getUser(1, (err, user) => {
  if (err) throw err;

  getOrders(user.id, (err, orders) => {
    if (err) throw err;

    orders.forEach((order) => {
      getOrderDetails(order.id, (err, details) => {
        if (err) throw err;

        processPayment(details, (err, payment) => {
          if (err) throw err;

          sendConfirmation(payment, (err, confirmation) => {
            if (err) throw err;
            console.log('Готово!');
          });
        });
      });
    });
  });
});

```

```
});
```

Проблемы Callback Hell:

1.  Глубокая вложенность (читаемость страдает)
 2.  Трудно обрабатывать ошибки на каждом уровне
 3.  Сложно управлять потоком (параллельность, последовательность)
 4.  Трудно отлаживать (стек вызовов теряется)
 5.  Трудно переиспользовать код
-

4. Промисы (Promises) в Node.js

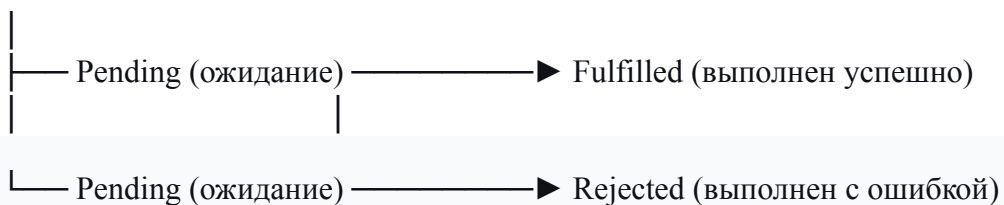
4.1. Что такое Promise?

Promise — это объект, представляющий результат асинхронной операции, которая может быть выполнена (успешно или с ошибкой) в будущем.

4.2. Состояния Promise

text

Promise



javascript

```
// Создание Promise
```

```
const myPromise = new Promise((resolve, reject) => {
  // Асинхронная операция
  const success = true;

  if (success) {
    resolve('Операция выполнена успешно!');
  } else {
    reject(new Error('Операция не удалась'));
  }
});
```

```
});
```

// Использование Promise

```
myPromise
```

```
.then((result) => {  
  console.log('Успех:', result);  
})  
.catch((error) => {  
  console.error('Ошибка:', error.message);  
});
```

4.3. Создание промисов

Пример 1: Чтение файла через Promise

```
javascript
```

```
function readFilePromise(path, encoding = 'utf8') {  
  return new Promise((resolve, reject) => {  
    fs.readFile(path, encoding, (err, data) => {  
      if (err) {  
        reject(err);  
      } else {  
        resolve(data);  
      }  
    });  
  });  
}
```

// Использование

```
readFilePromise('file.txt', 'utf8')  
  .then(data => console.log('Содержимое:', data))  
  .catch(err => console.error('Ошибка:', err.message));
```

Пример 2: HTTP-запрос через Promise

```
javascript
```

```
function fetchData(url) {  
  return new Promise((resolve, reject) => {  
    https.get(url, (res) => {  
      let data = "";
```

```

    res.on('data', (chunk) => data += chunk);
    res.on('end', () => {
      try {
        resolve(JSON.parse(data));
      } catch (err) {
        reject(err);
      }
    });
  }).on('error', reject);
});
}

// Использование
fetchData('https://api.example.com/data')
  .then(data => console.log('Получены данные:', data))
  .catch(err => console.error('Ошибка запроса:', err.message));

```

4.4. Цепочки промисов (Chaining)

```
javascript
```

```

function getUser(id) {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ id, name: 'Alice' }), 1000);
  });
}

function getProfile(userId) {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ userId, age: 25, city: 'Moscow' }), 1000);
  });
}

function getSettings(userId) {
  return new Promise((resolve) => {
    setTimeout(() => resolve({ userId, theme: 'dark' }), 1000);
  });
}

// Цепочка промисов
getUser(1)

```

```

.then(user => {
  console.log('Пользователь:', user);
  return getProfile(user.id);
})
.then(profile => {
  console.log('Профиль:', profile);
  return getSettings(profile.userId);
})
.then(settings => {
  console.log('Настройки:', settings);
  console.log('Все данные загружены!');
})
.catch(error => {
  console.error('Ошибка:', error.message);
});

```

4.5. Параллельные операции с Promise

Promise.all() — ожидает выполнения ВСЕХ промисов:

javascript

```

const promises = [
  getUser(1),
  getUser(2),
  getUser(3)
];

```

Promise.all(promises)

```

.then(users => {
  console.log('Все пользователи:', users);
})
.catch(error => {
  console.error('Ошибка:', error.message);
});

```

Promise.race() — возвращает результат первого завершённого:

javascript

```

const promise1 = new Promise(resolve => setTimeout(resolve, 1000, 'Первый'));
const promise2 = new Promise(resolve => setTimeout(resolve, 500, 'Второй'));

```

Promise.race([promise1, promise2])

```
.then(result => console.log('Победитель:', result)); // 'Второй'
```

Promise.allSettled() — ждёт все промисы, игнорируя ошибки:

javascript

```
const promises = [  
  Promise.resolve('Успех 1'),  
  Promise.reject('Ошибка 2'),  
  Promise.resolve('Успех 3')  
];
```

```
Promise.allSettled(promises)  
  .then(results => {  
    results.forEach((result, index) => {  
      if (result.status === 'fulfilled') {  
        console.log(`Задача ${index}:`, result.value);  
      } else {  
        console.log(`Задача ${index} ошибка:`, result.reason);  
      }  
    });  
  });
```

5. Преобразование колбэков в промисы с использованием util.promisify

5.1. Что такое promisify?

util.promisify() — это встроенный метод Node.js, который преобразует функцию с error-first колбэком в функцию, возвращающую Promise.

5.2. Базовое использование

javascript

```
const util = require('util');  
const fs = require('fs');
```

```

// Преобразуем fs.readFile в промис-версию
const readFile = util.promisify(fs.readFile);

// Теперь можно использовать с async/await или .then()
async function readConfig() {
  try {
    const data = await readFile('config.json', 'utf8');
    return JSON.parse(data);
  } catch (error) {
    console.error('Ошибка чтения конфига:', error.message);
    return null;
  }
}

// Или с .then()
readFile('file.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error(err));

```

5.3. Промисификация нескольких методов

```

javascript

const util = require('util');
const fs = require('fs');

// Преобразуем несколько методов сразу
const readFile = util.promisify(fs.readFile);
const writeFile = util.promisify(fs.writeFile);
const mkdir = util.promisify(fs.mkdir);
const stat = util.promisify(fs.stat);

// Использование
async function createFileWithDir(path, content) {
  try {
    // Проверяем существование директории
    try {
      await stat(path);
      console.log('Директория существует');
    } catch (err) {
      // Создаём директорию
      await mkdir(path, { recursive: true });
    }
  }
}

```

```

        console.log('Директория создана');
    }

    // Записываем файл
    await writeFile(`${path}/data.txt`, content);
    console.log('Файл создан');
} catch (error) {
    console.error('Ошибка:', error.message);
}
}

```

```
createFileWithDir('./new_folder', 'Hello, World!');
```

5.4. Промисификация пользовательских функций

javascript

```

const util = require('util');

// Пользовательская функция с error-first колбэком
function calculateSquare(number, callback) {
    if (typeof number !== 'number') {
        callback(new Error('Аргумент должен быть числом'), null);
        return;
    }
    callback(null, number * number);
}

// Преобразуем в промис
const calculateSquareAsync = util.promisify(calculateSquare);

// Использование
calculateSquareAsync(5)
    .then(result => console.log('Квадрат 5 =', result))
    .catch(err => console.error('Ошибка:', err.message));

// Попытка с ошибкой
calculateSquareAsync('text')
    .then(result => console.log(result))
    .catch(err => console.error('Ошибка:', err.message));

```

5.5. Важные нюансы promisify

1. Промисификация не работает с функциями, использующими `this`

javascript

```
class Database {
  query(sql, callback) {
    // this используется внутри
    setTimeout(() => callback(null, `Result of: ${sql}`), 100);
  }
}

const db = new Database();
// Не будет работать, так как теряется контекст this
const query = util.promisify(db.query);
query('SELECT * FROM users').catch(err => console.error(err));

// Правильный способ
const queryBound = util.promisify(db.query.bind(db));

queryBound('SELECT * FROM users').then(result => console.log(result));
```

2. Промисификация методов с несколькими результатами

javascript

```
function multipleResults(callback) {
  callback(null, 'result1', 'result2', 'result3');
}

// promisify вернёт только первый результат
const multiAsync = util.promisify(multipleResults);
multiAsync().then(result => console.log(result)); // 'result1'

// Решение: использовать объект или массив
function multipleResultsFixed(callback) {
  callback(null, { r1: 'result1', r2: 'result2', r3: 'result3' });
}

const multiAsyncFixed = util.promisify(multipleResultsFixed);

multiAsyncFixed().then(result => console.log(result.r1, result.r2, result.r3));
```

6. Асинхронные функции (Async/Await) в Node.js

6.1. Синтаксис async/await

async — объявляет асинхронную функцию, которая всегда возвращает Promise

await — приостанавливает выполнение функции до выполнения Promise

javascript

// Базовый пример

```
async function fetchUserData() {  
  // Код выполняется асинхронно  
  return { id: 1, name: 'Alice' };  
}
```

// Эквивалентно:

```
function fetchUserDataPromise() {  
  return Promise.resolve({ id: 1, name: 'Alice' });  
}
```

// Использование

```
fetchUserData().then(user => console.log(user));
```

6.2. Работа с await

javascript

```
function delay(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}
```

```
async function greetUser() {  
  console.log('Начинаем...');  
  await delay(2000); // Ждём 2 секунды  
  console.log('Прошло 2 секунды');  
  return 'Привет!';  
}
```

// Использование

```
greetUser().then(message => console.log(message));
```

// Можно использовать только внутри async

```
(async () => {  
  const result = await greetUser();  
  console.log('Результат:', result);  
})();
```

6.3. Реальный пример: чтение нескольких файлов

javascript

```
const fs = require('fs');  
const util = require('util');  
  
const readFile = util.promisify(fs.readFile);  
  
async function readMultipleFiles() {  
  try {  
    console.log('Начинаем чтение...');  
  
    // Последовательное чтение  
    const data1 = await readFile('file1.txt', 'utf8');  
    console.log('Файл 1:', data1);  
  
    const data2 = await readFile('file2.txt', 'utf8');  
    console.log('Файл 2:', data2);  
  
    const data3 = await readFile('file3.txt', 'utf8');  
    console.log('Файл 3:', data3);  
  
    console.log('Все файлы прочитаны!');  
    return { data1, data2, data3 };  
  } catch (error) {  
    console.error('Ошибка чтения:', error.message);  
    throw error;  
  }  
}  
  
// Параллельное чтение с async/await  
async function readMultipleFilesParallel() {  
  try {  
    console.log('Начинаем параллельное чтение...');  
  
    const [data1, data2, data3] = await Promise.all([
```

```

        readFile('file1.txt', 'utf8'),
        readFile('file2.txt', 'utf8'),
        readFile('file3.txt', 'utf8')
    ]);

    console.log('Все файлы прочитаны параллельно!');
    return { data1, data2, data3 };
} catch (error) {
    console.error('Ошибка:', error.message);
    throw error;
}
}

```

6.4. Лучшие практики использования async/await

1. Избегайте чрезмерного использования await в циклах

javascript

//  Плохо: последовательное выполнение (медленно)

```

const userIds = [1, 2, 3, 4, 5];
for (const id of userIds) {
    const user = await getUser(id);
    console.log(user);
}

```

//  Хорошо: параллельное выполнение (быстро)

```

const userPromises = userIds.map(id => getUser(id));
const users = await Promise.all(userPromises);

```

```

console.log(users);

```

2. Не забывайте про обработку ошибок

javascript

//  Плохо: ошибка остановит всё выполнение

```

async function processAll() {
    const data = await fetchData(); // Если здесь ошибка, всё упадёт
    return processData(data);
}

```

//  Хорошо: обрабатываем ошибки

```

async function processAllSafe() {
  try {
    const data = await fetchData();
    return processData(data);
  } catch (error) {
    console.error('Ошибка обработки:', error.message);
    return null; // или выбросить свою ошибку
  }
}

```

3. Используйте IIFE для верхнего уровня

javascript

```

// В верхнем уровне кода await недоступен
// ❌ Ошибка: await is only valid in async functions
// const data = await fetchData();

```

```

// ✅ Правильно: используем IIFE
(async () => {
  try {
    const data = await fetchData();
    console.log('Данные:', data);
  } catch (error) {
    console.error('Ошибка:', error);
  }
})();

```

```

// Или используем .then()
fetchData()
  .then(data => console.log(data))
  .catch(err => console.error(err));

```

7. Обработка ошибок при работе с асинхронным кодом

7.1. Обработка ошибок в колбэках

javascript

```

const fs = require('fs');

// Стандартная обработка в error-first колбэках
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) {
    // Проверяем тип ошибки
    if (err.code === 'ENOENT') {
      console.error('Файл не найден');
    } else {
      console.error('Ошибка чтения:', err.message);
    }
  }
  return;
}
console.log('Данные:', data);
});

```

7.2. Обработка ошибок в промисах

```

javascript

// Цепочка промисов
fetchUser(1)
  .then(user => fetchProfile(user.id))
  .then(profile => fetchSettings(profile.userId))
  .then(settings => {
    console.log('Все данные:', settings);
  })
  .catch(error => {
    // Ловит любую ошибку в цепочке
    console.error('Ошибка:', error.message);

    // Можно пробросить свою ошибку
    throw new Error('Критическая ошибка');
  })
  .finally(() => {
    // Выполняется всегда (очистка ресурсов)
    console.log('Операция завершена');
  });

```

7.3. Обработка ошибок в async/await

javascript

// Try/catch блок

```
async function loadUserData() {  
  try {  
    const user = await fetchUser(1);  
    const profile = await fetchProfile(user.id);  
    const settings = await fetchSettings(profile.userId);  
  
    return { user, profile, settings };  
  } catch (error) {  
    console.error('Ошибка загрузки:', error.message);  
  
    // Возвращаем значение по умолчанию  
    return { user: null, profile: null, settings: null };  
  }  
}
```

// Отлавливаем ошибки на уровне вызова

```
loadUserData()  
  .then(data => {  
    if (data.user === null) {  
      console.log('Используем данные по умолчанию');  
    }  
  })  
  .catch(err => {  
    // Если ошибка не была обработана внутри  
    console.error('Необработанная ошибка:', err);  
  
  });
```

7.4. Глобальная обработка ошибок

javascript

// Обработка непоиманных ошибок в промисах

```
process.on('unhandledRejection', (reason, promise) => {  
  console.error('Необработанная ошибка Promise:', reason);  
  console.error('В контексте:', promise);  
  
  // Логирование в файл  
  fs.appendFileSync('error.log',  
    `[${new Date().toISOString()}] Unhandled Rejection: ${reason}\n`  
  );  
});
```

```

);

// В продакшине можно перезапустить приложение
// process.exit(1);
});

// Обработка синхронных ошибок
process.on('uncaughtException', (error) => {
  console.error('Необработанное исключение:', error);
  console.error('Stack trace:', error.stack);

  // Сохраняем стек для отладки
  fs.writeFileSync('crash.log',
    `[$${new Date().toISOString()}] ${error.stack}\n`
  );

  // В продакшине лучше перезапустить процесс
  process.exit(1);
});

```

7.5. Создание собственных ошибок

javascript

```

// Кастомные ошибки
class DatabaseError extends Error {
  constructor(message, query) {
    super(message);
    this.name = 'DatabaseError';
    this.query = query;
    this.timestamp = new Date();
  }
}

class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
    this.timestamp = new Date();
  }
}

```

// Использование

```
async function getUserFromDB(id) {
  try {
    if (typeof id !== 'number') {
      throw new ValidationError('ID должен быть числом', 'id');
    }

    const user = await db.query(`SELECT * FROM users WHERE id = ${id}`);

    if (!user) {
      throw new DatabaseError(
        `Пользователь с ID ${id} не найден`,
        `SELECT * FROM users WHERE id = ${id}`
      );
    }

    return user;
  } catch (error) {
    if (error instanceof ValidationError) {
      console.error('Ошибка валидации:', error.message);
    } else if (error instanceof DatabaseError) {
      console.error('Ошибка БД:', error.message);
      console.error('Запрос:', error.query);
    } else {
      console.error('Неизвестная ошибка:', error);
    }
    throw error;
  }
}
```

7.6. Паттерны повторных попыток (Retry)

javascript

```
async function retryOperation(fn, maxAttempts = 3, delay = 1000) {
  let lastError;

  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      console.log(`Попытка ${attempt} из ${maxAttempts}`);
      return await fn();
    } catch (error) {
      lastError = error;
    }
  }

  throw lastError;
}
```

```

    } catch (error) {
      lastError = error;
      console.log(`Ошибка попытки ${attempt}:`, error.message);

      if (attempt === maxAttempts) {
        break;
      }

      console.log(`Ждём ${delay}мс перед следующей попыткой...`);
      await new Promise(resolve => setTimeout(resolve, delay));
    }
  }

  throw new Error(`Не удалось выполнить операцию после ${maxAttempts} попыток.
Последняя ошибка: ${lastError.message}`);
}

```

// Использование

```

async function fetchWithRetry() {
  try {
    const data = await retryOperation(
      async () => {
        const response = await fetch('https://api.example.com/data');
        if (!response.ok) {
          throw new Error(`HTTP ${response.status}`);
        }
        return response.json();
      },
      3, // 3 попытки
      2000 // задержка 2 секунды
    );
    console.log('Данные получены:', data);
  } catch (error) {
    console.error('Операция не удалась:', error.message);
  }
}

```

7.7. Timeout для асинхронных операций

javascript

// Функция с таймаутом

```

function withTimeout(promise, timeoutMs) {
  return new Promise((resolve, reject) => {
    const timer = setTimeout(() => {
      reject(new Error(`Операция превысила время ${timeoutMs}мс`));
    }, timeoutMs);

    promise
      .then(result => {
        clearTimeout(timer);
        resolve(result);
      })
      .catch(error => {
        clearTimeout(timer);
        reject(error);
      });
  });
}

```

// Использование

```

async function fetchWithTimeout() {
  try {
    const data = await withTimeout(
      fetchData('https://api.example.com'),
      5000 // 5 секунд таймаут
    );
    console.log('Данные загружены:', data);
  } catch (error) {
    if (error.message.includes('превысил')) {
      console.error('Таймаут:', error.message);
    } else {
      console.error('Ошибка:', error.message);
    }
  }
}

```

8. Сравнительная таблица подходов

Характеристика	Callbacks	Promises	Async/Await
Читаемость	Плохая (вложенность)	Хорошая (цепочки)	Отличная (синхронный вид)
Обработка ошибок	Сложная (проверка err)	Хорошая (.catch())	Отличная (try/catch)
Вложенность	Высокая	Средняя	Низкая
Параллельность	Сложная	Простая (Promise.all)	Простая (Promise.all)
Отладка	Трудная	Средняя	Хорошая
Количество кода	Много	Среднее	Мало
Совместимость	Везде	ES6+	ES8+ (Node 8+)
Рекомендация	 Устаревший	 Для сложной логики	  Основной подход